

Feature Scaling: Why It's Important

Feature scaling is a technique used to standardize the range of independent variables or features of data. It's a crucial preprocessing step that can significantly speed up the convergence of gradient descent.

The Problem: Features with Different Scales

Machine learning models often use multiple features, and these features can have vastly different ranges of values.

- **Example:** Predicting house prices using two features:
 - **Size (x_1):** Ranges from 300 to 2,000 sq ft.
 - **Number of Bedrooms (x_2):** Ranges from 0 to 5.

When features have such different scales, the parameters (w) associated with them will also tend to have very different scales. A feature with a large range (like size) will likely have a small weight (w_1), while a feature with a small range (like bedrooms) will likely have a large weight (w_2) to balance their respective impacts on the prediction.

The Impact on Gradient Descent

This imbalance in feature scales has a direct, negative impact on the performance of gradient descent.

- **Elongated Contour Plots:** The cost function's contour plot becomes very tall and skinny, forming elongated ovals (ellipses). This is because a small change in a small parameter (like w_1) can cause a massive change in the cost, while a large change is needed in a large parameter (like w_2) to have a similar effect.
 - **Inefficient Convergence:** Because of this skewed shape, gradient descent tends to **bounce back and forth** inefficiently, taking a long, indirect path to find the global minimum. This significantly slows down the training process.
-

The Solution: Scaling Features

By scaling the features, we transform them so they all have a comparable range of values (e.g., from 0 to 1).

- **Circular Contour Plots:** When features are scaled, the cost function's contour plot becomes more symmetrical and circular.
- **Faster Convergence:** With a more circular contour plot, gradient descent can find a much

more **direct path** to the global minimum. The algorithm no longer needs to oscillate and can converge much more quickly.

Feature Scaling Methods

There are several common techniques to scale features so they fall within a similar range, which helps gradient descent converge faster.

1. Max Scaling

This is one of the simplest methods. You just divide each feature by its maximum value in the training set.

- **Formula:** $x_{scaled} = \frac{x_{original}}{max(x)}$
 - **Resulting Range:** The scaled feature will typically range from just above 0 to 1.
-

2. Mean Normalization

This method centers the feature values around zero.

- **Process:**
 1. Calculate the **mean** (average), μ , of the feature in the training set.
 2. Calculate the **range** of the feature (i.e., $max(x) - min(x)$).
 - **Formula:** $x_{scaled} = \frac{x_{original} - \mu}{max(x) - min(x)}$
 - **Resulting Range:** The scaled feature will be centered around 0.
-

3. Z-score Normalization

This is a very common technique that scales features based on their mean and standard deviation.

- **Process:**
 1. Calculate the **mean**, μ , of the feature.
 2. Calculate the **standard deviation**, σ , of the feature.
 - **Formula:** $x_{scaled} = \frac{x_{original} - \mu}{\sigma}$
 - **Resulting Range:** The scaled feature will have a mean of 0 and a standard deviation of 1.
-

Rule of Thumb

The general goal of feature scaling is to get your features into a range of roughly **-1 to +1**.

- **Acceptable Ranges:** Ranges like **-3 to +3** or **-0.3 to +0.3** are perfectly fine and don't necessarily need scaling.

- **Ranges to Scale:** You should consider scaling features with very large or very small ranges, as these can slow down gradient descent.
 - **Too Large:** A range like **-100 to +100**.
 - **Too Small:** A range like **-0.001 to +0.001**.

When in doubt, it's almost never harmful to perform feature scaling.

Checking for Convergence

When you run gradient descent, it's important to monitor its progress to ensure it's actually working and approaching the minimum cost.

The Learning Curve

The most common way to check for convergence is to plot a **learning curve**.

- **What it is:** A graph of the cost function, $J(w, b)$, versus the **number of iterations** of gradient descent.
 - **Horizontal axis:** Number of iterations.
 - **Vertical axis:** Cost J .
 - **How to Interpret It:**
 - **If gradient descent is working correctly:** The cost J should **decrease** after every single iteration. If the cost ever *increases*, it's a sign that your learning rate α is likely too large, or there might be a bug in your code.
 - **When has it converged?** The algorithm has **converged** when the learning curve flattens out. This means that additional iterations are no longer significantly decreasing the cost.
 - **Important Note:** The number of iterations needed for convergence can vary greatly between different problems (from dozens to hundreds of thousands). The learning curve is the best way to visualize this for your specific application.
-

Automatic Convergence Test

An alternative to visual inspection is an **automatic convergence test**.

- **How it works:** You choose a very small number, called **epsilon** (ϵ), for example, 0.001. If the cost J decreases by less than epsilon in a single iteration, you can declare that the algorithm has converged.
- **Practicality:** While this method exists, it can be difficult to choose the right threshold for epsilon. Visually inspecting the learning curve is often a more reliable and insightful approach.

Choosing the Learning Rate (α)

Selecting a good learning rate is critical for the performance of gradient descent. An incorrect choice can lead to slow training or a complete failure to converge.

How to Tell if Your Learning Rate is Incorrect

You can diagnose issues with your learning rate by examining the **learning curve** (J vs. number of iterations).

- **If α is too large:**
 - The cost J might **increase and decrease** (oscillate) over iterations. This happens when the algorithm repeatedly **overshoots** the minimum.
 - In a more extreme case, the cost J might **consistently increase** on every iteration. This is a clear sign of **divergence**.
 - **Solution:** In both cases, the fix is to **try a smaller learning rate α** .
 - **If α is too small:**
 - The cost J will decrease consistently, but **very slowly**.
 - **Solution:** While this will eventually work, it's inefficient. You should try a larger value for α to speed up convergence.
-

A Method for Choosing α

A practical approach is to test a range of α values and observe which one performs best.

1. **Start with a range of values:** It's common to try values on a logarithmic scale, often increasing by a factor of approximately 3 each time.
 - **Example values to try:** 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1, etc.
 2. **Test and Observe:** For each value of α , run gradient descent for a moderate number of iterations and plot the learning curve.
 3. **Analyze the Curves:**
 - Identify values that are **too large** (cost diverges or oscillates).
 - Identify values that are **too small** (cost decreases too slowly).
 4. **Select the Best α :** Choose the learning rate that causes the cost to **decrease quickly and consistently**. Often, the best choice is the largest value that works without diverging, or one slightly smaller than that.
-

Debugging Tip 🐛

If your cost J is not decreasing on every iteration, it's important to distinguish between a bad learning rate and a bug in your code.

- **Test:** Set α to a **very small number** (e.g., 0.0001).
- **Result:**
 - If J now decreases on every iteration (even if slowly), your original α was too large.
 - If J *still* increases or oscillates, you likely have a **bug in your code** (e.g., using + instead of - in the update rule).

Feature Engineering

Feature engineering is the process of using your domain knowledge or intuition about a problem to create new features from the original ones. This is often done by transforming or combining existing features to help the learning algorithm make more accurate predictions. The choice of features can have a huge impact on your model's performance.

An Example: Housing Prices

Let's say you're predicting house prices and have two features:

- x_1 : The width (or frontage) of the plot of land.
- x_2 : The depth of the plot of land.

Initial Model

You could start with a standard linear model using these two features:

$$f(x) = w_1x_1 + w_2x_2 + b$$

This model might work reasonably well.

Engineering a New Feature

However, you might have an intuition that the total **area** of the land is a more powerful predictor of price than its width and depth treated separately.

1. **Create a new feature:** You can define a new feature, x_3 , that represents the area.

$$x_3 = x_1 \times x_2$$

2. **Build a new model:** You can now create a more sophisticated model that includes the original features and your new, engineered feature.

$$f(x) = w_1x_1 + w_2x_2 + w_3x_3 + b$$

This new model is more flexible. It allows the learning algorithm to determine the relative importance of width, depth, and the total area, potentially leading to a much better fit for the data. By defining new features, you can often create a much more effective model.

Polynomial Regression: Fitting Curves

Polynomial regression is a powerful technique that combines multiple linear regression with **feature engineering** to fit complex, **non-linear curves** to the data. It's used when a simple straight line is not a good fit for the underlying relationship between the features and the target.

Creating Polynomial Features

The core idea is to create new features by raising an original feature to different powers.

- **Example:** For a single feature x (like house size), you can create new features to build more complex models:
 - **Quadratic Model:** Use x and x^2 . The model becomes $f(x) = w_1x + w_2x^2 + b$. This can fit a parabolic curve.
 - **Cubic Model:** Use x , x^2 , and x^3 . The model becomes $f(x) = w_1x + w_2x^2 + w_3x^3 + b$. This can fit a more complex S-shaped curve.
-

The Importance of Feature Scaling

When you use polynomial regression, **feature scaling becomes extremely important**.

- **Reason:** The ranges of the engineered features can become drastically different.
 - If x (size) ranges from 1 to 1,000...
 - Then x^2 (size squared) ranges from 1 to 1,000,000.
 - And x^3 (size cubed) ranges from 1 to 1,000,000,000.
 - **Impact:** Without scaling, these vastly different ranges will cause gradient descent to converge very slowly and inefficiently.
-

Other Feature Choices

You are not limited to just polynomial terms. You can engineer other non-linear features based on your understanding of the data.

- **Example:** You could use the **square root** of a feature.
 - **Model:** $f(x) = w_1x + w_2\sqrt{x} + b$
 - The square root function increases but at a decreasing rate, which might be a suitable model for some datasets.

How to Choose Features: Deciding which features to create or use is a key part of building a good model. Later in the specialization, you will learn systematic ways to evaluate and select the best features for your problem.